



SE116 - Introduction to Programming with Java II
Spring 2025-2026 Semester Project

Dr. Erdem OKUR

ObjectVille II Project Report

Group MEDA

Mert KOÇYİĞİT

20210601039

Elif Eren ÇARIKCI

20240601023

Deren KOCABAŞ

20240601055

Arda ABACI

20240601001

Abstract

This report documents the architectural design, engineering specifications, and object-oriented implementation of ObjectVille II, a grid-based city simulation engine. The program is written purely in Java and packaged as a modular, executable framework. Its primary engineering goal is to model a discrete two-dimensional city map, read from a plain-text file, and to run a fixed number of simulation ticks that distribute utilities, services, and economic resources across the grid while logging every event.

The system models a 2D grid of distinct cell types: resource consumers (Housing, Commercial, and Industrial zones), public service entities (Hospitals, Police Stations, and Schools), structural transport links (Road segments), core utility infrastructure (Power Plants, Water Stations, and Internet Hubs), and inert background tiles (EmptyCell). Every map element derives from a single abstract Cell base class, enabling fully polymorphic handling of the grid.

By leveraging a decoupled, multi-package architecture, two small functional interfaces (Connectable and Producible), and an enums package that defines the three resource, utility, and service vocabularies, the software keeps a clean separation of concerns. The execution engine runs synchronous tick cycles: utilities are routed from each provider through contiguous connectable cells using a breadth-first search (BFS), services are applied within a radius computed by Euclidean distance, economic resources are shared from a global pool among consuming zones, and each zone then updates its level and production. The map loader performs strict input validation, rejecting unknown symbols and ragged grids without corrupting the simulation. Ultimately, ObjectVille II demonstrates how abstract programming concepts such as polymorphism, interface segregation, and graph traversal translate into a stable, readable software engine.

1. Introduction

Modern simulation software relies heavily on robust data abstraction to model real-world logistical patterns, resource networks, and physical constraints. Simulating an interconnected city environment is an excellent benchmark for evaluating complex object relationships. ObjectVille II brings these paradigms into a flexible, two-dimensional discrete grid environment that is built once from a map file and then advanced through a fixed number of ticks.

At its core, the application is organized around four foundational layers:

- **The cell domain layer:** Houses all physical grid assets. Every layout component — whether a residential block, a road segment, or a power plant — derives from a single abstract base class, `Cell`. This structure enables full polymorphic handling across the grid matrix.
- **The interfaces layer:** Defines two narrow behavioral contracts. `Connectable` expresses whether a cell can carry utility flow during pathfinding, and `Producible` expresses that a cell can generate resources. Isolating these actions keeps an object's runtime behavior separate from its class taxonomy.
- **The enums layer:** Defines the closed vocabularies used throughout the engine — `ResourceType` (population, goods, lifestyle), `UtilityType` (electricity, water, internet), and `ServiceType` (security, health, education). These enums are imported by nearly every other package and act as the shared language between producers and consumers.
- **The city orchestration layer:** Holds the grid and drives time-based state updates. The `City` class loads and stores the map; the `Simulation` class runs the tick loop, performs utility routing, applies services, shares resources, and prints all events.

Beyond fulfilling its academic mandate for the SE116 curriculum, ObjectVille II is engineered for clean maintainability. It prevents runtime errors through strict map-file validation and forms a reusable, extensible foundation for future optimization tasks, automated test harnesses, and graphical visualization engines.

2. Problem Statement

Simulating dynamic resource distribution and municipal service coverage on a discrete grid introduces real architectural challenges. Naive simulation systems often couple individual cells tightly to their neighbors, creating rigid dependency structures. The fundamental problem addressed by ObjectVille II is the need for a decoupled, readable grid environment where utility flows (power, water, data), service coverage, and economic resources can interact across the map without corrupting global data states.

More specifically, the system must overcome three main challenges:

1. **Topological Resource Routing:** Power plants, water stations, and internet hubs must reach the zones that need them. Rather than scanning the whole map, each provider performs a breadth-first search outward through contiguous connectable cells (roads and zones), delivering its utility only to zones that require that utility type and stopping when its capacity is exhausted.
2. **Radial Service Coverage:** Hospitals, police stations, and schools have a fixed effective radius. The engine evaluates the Euclidean distance between a provider and each zone, and grants the corresponding service (health, security, or education) to every zone that falls within range and is eligible for it.
3. **Polymorphic, Type-Driven Construction:** The grid is built from a character map in which each symbol maps to a concrete Cell subclass. The loader must construct the correct type for every symbol, group zones, utilities, and services into separate lists for efficient iteration, and reject any unknown symbol or malformed grid before the simulation begins.

By organizing the grid around an interface-driven design and a centralized simulation engine, ObjectVille II keeps these concerns separate and bridges the gap between theoretical software patterns and a practical, fault-tolerant grid simulation.

3. System Requirements

3.1 Non-Functional Requirements (NFR)

The technical constraints and quality metrics governing the development of ObjectVille II are summarized below.

Requirement ID	Technical Specification	Engineering Implementation Status
NFR1	Pure Java Implementation	Developed entirely in object-oriented Java (uses modern features such as switch expressions, pattern matching for instanceof, and EnumMap/EnumSet). No external proprietary libraries are required.
NFR2	Version Control & Tracking	Managed within a private repository with clean, incremental commits tracking feature additions and class splits by Team MEDA.
NFR3	Deployment Packaging	Built via build.sh / build.bat using javac and jar, with manifest.mf declaring 'Main-Class: ObjectVilleGame', producing a portable archive named ObjectVilleGame.jar.
NFR4	Detailed Technical Reporting	Documented with class descriptions, real method and field signatures, execution flow, code screenshots, and verified console outputs.

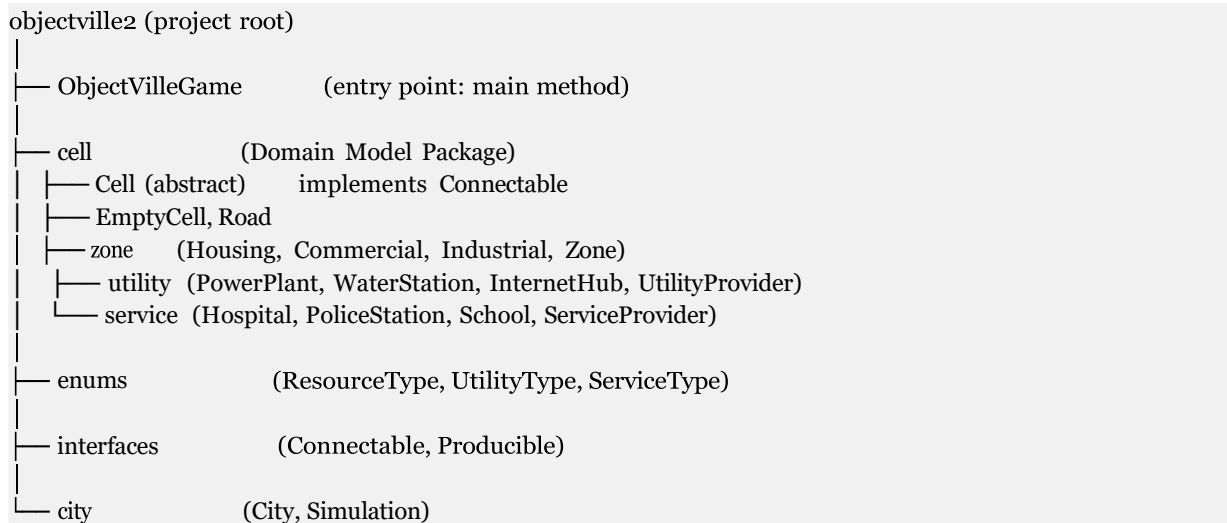
3.2 Functional Requirements (FR)

- **FR1 - Map-File Grid Construction:** On startup the City class reads a text map file, skipping blank lines and lines starting with '#', optionally honouring a leading 'rows cols' header line, and instantiates a uniform two-dimensional Cell[][] grid before the simulation runs.
- **FR2 - Polymorphic Cell Construction:** Each map symbol is translated to a concrete subtype (Housing, Industrial, Commercial, PowerPlant, WaterStation, InternetHub, PoliceStation, Hospital, School, Road, EmptyCell) through a single factory switch, with unknown symbols raising a clear IOException.
- **FR3 - Pathfinding Connectivity via Connectable Cells:** Roads and zones are Connectable and act as the carriers of utility flow. Each utility provider routes its output through contiguous connectable cells using BFS, so separated map sectors are only served if a connected path exists.
- **FR4 - Production & Resource Sharing:** Zones implement Producible. After receiving utilities and services, each zone updates its level and produces a resource (Housing → population, Industrial → goods, Commercial → lifestyle). Produced output accumulates into a global resource pool that is shared evenly among consuming zones on the next tick.
- **FR5 - Clock-Tick State Tracking:** The Simulation class runs a configurable number of ticks. Each tick resets per-tick data, distributes services and utilities, shares resources, updates zone levels, and logs every allocation, generation, and level change to the chosen output stream.

4. Architecture and Design

4.1 High-Level Component Breakdown

ObjectVille II separates systems by package to enforce clear boundaries between data definitions and logical flow:



- **cell package:** The primary domain model. It encapsulates coordinates (row, col) and type-specific behavior, isolating object definitions from the simulation loop. The abstract Cell implements Connectable; its zone, utility, and service sub-packages hold the consumer, generator, and municipal hierarchies.
- **enums package:** Declares the three closed vocabularies (ResourceType, UtilityType, ServiceType), each carrying a lowercase display label used in log output.
- **interfaces package:** Declares the two functional contracts that decouple behavior from class identity: Connectable (can this cell carry flow?) and Producible (can this cell generate resources?).
- **city package:** Houses the central logic. City builds and stores the grid and exposes neighbor and radius queries; Simulation runs the tick loop, performs routing, and writes all output.

4.2 Component Interactions and Data Flow

During a single tick, data moves across the architecture in an ordered pipeline:

```
[reset zone tick data] -> [distribute services] -> [distribute utilities (BFS)]  
-> [distribute pooled resources] -> [update zones & produce] -> [accumulate production]
```

- **Step 1 — Services:** Each ServiceProvider grants its service to every eligible zone within its radius (Euclidean distance $\leq$ radius).
- **Step 2 — Utilities:** Each UtilityProvider runs a BFS from its own coordinate through connectable cells, delivering its utility to requiring zones up to a fixed capacity (100 units).
- **Step 3 — Resources:** Resources accumulated in the global pool from the previous tick are divided evenly among the zones that consume each resource type.
- **Step 4 — Update & Produce:** Each zone recomputes its level from the utilities and services it received, produces its output resource, and logs any level change. The new output is accumulated into the pool for the next tick.

4.3 Design Patterns and Principles

- **Interface Segregation Principle (ISP):** Behavior is split into two narrow interfaces rather than one large contract. A cell becomes part of the routing network simply by returning true from isConnectable(), and becomes a producer by implementing Producible — neither forces unused methods onto unrelated classes.
- **Single Responsibility Principle (SRP):** Classes derived from Cell manage only their own state (coordinates, level, received utilities/services). All cross-cell work — routing, service projection, resource sharing — is handled exclusively by the City and Simulation classes in the city package.
- **Polymorphism & the Factory Switch:** City.createCell uses a single switch expression to turn map symbols into the correct Cell subtype, and the rest of the engine operates on Cell, Zone, UtilityProvider, and ServiceProvider references without caring about concrete types.

5. Implementation Details

5.1 Deep Package and Class Breakdown

5.1.1 The cell Package (Foundational Hierarchy)

- **Cell (Abstract Base Class):** The root ancestor of every grid element. It implements Connectable.
 - Fields: final int row, final int col.
 - Methods: getRow(), getCol(), abstract getSymbol(), isConnectedable() (returns false by default), getDisplaySymbol().
- **EmptyCell (Concrete Class):** Extends Cell. A safe placeholder for unused tiles; its symbol is 'E' and it is not connectable.
- **Road (Concrete Class):** Extends Cell. Symbol 'R'. Overrides isConnectedable() to return true, forming the carrier network for utility BFS. It has no extra fields.

```
1 package cell;
2
3 import interfaces.Connectable;
4
5 public abstract class Cell implements Connectable {
6     private final int row;
7     private final int col;
8
9     protected Cell(int row, int col) {
10         this.row = row;
11         this.col = col;
12     }
13
14     public int getRow() { return row; }
15
16     public int getCol() { return col; }
17
18     public abstract char getSymbol();
19
20     @Override
21     public boolean isConnectedable() { return false; }
22
23     public String getDisplaySymbol() { return Character.toString(getSymbol()); }
24 }
```

Figure 1: The abstract Cell base class.

5.1.2 The cell.zone Sub-Package (Resource Consumers)

- **Zone (Abstract Base Class):** Extends Cell and implements Producible. Tracks a level (0–3), a demand value, and EnumMap/EnumSet records of utilities, services, and resources received during the current tick. It is connectable, and its display symbol appends its level (e.g. 'H2').
 - Fields: int level, int demand, plus EnumMaps for utilities/resources received and an EnumSet of services received.
 - Key methods: resetTickData(), receiveUtility(), receiveService(), receiveResource(), hasAllServices(), getMinimumRequiredUtility(), baseOutput(), update(); plus abstract getRequiredUtilities(), consumesResource(), calculateTargetLevel().
- **Housing (Concrete Class):** Extends Zone. Symbol 'H'. Requires electricity, water, and internet; consumes the LIFESTYLE resource; produces POPULATION.
- **Commercial (Concrete Class):** Extends Zone. Symbol 'C'. Requires electricity, water, and internet; consumes POPULATION and GOODS; produces LIFESTYLE.
- **Industrial (Concrete Class):** Extends Zone. Symbol 'I'. Requires electricity and water; consumes POPULATION; produces GOODS.

```

1 package cell.zone;
2
3 import enums.ResourceType;
4 import enums.ServiceType;
5 import enums.UtilityType;
6
7 import java.util.EnumSet;
8 import java.util.Map;
9 import java.util.Set;
10
11 public class Industrial extends Zone {
12     public Industrial(int row, int col) {
13         super(row, col);
14     }
15
16     @Override
17     public char getSymbol() {
18         return 'I';
19     }
20
21     @Override
22     public Set<UtilityType> getRequiredUtilities() {
23         return EnumSet.of(UtilityType.ELECTRICITY, UtilityType.WATER);
24     }
25
26     @Override
27     public boolean consumesResource(ResourceType type) {
28         return type == ResourceType.POPULATION;
29     }
30 }

```

Figure 2: The Industrial zone.

```

1 package cell.zone;
2
3 import enums.ResourceType;
4 import enums.UtilityType;
5
6 import java.util.EnumSet;
7 import java.util.Map;
8 import java.util.Set;
9
10 public class Housing extends Zone {
11     public Housing(int row, int col) {
12         super(row, col);
13     }
14
15     @Override
16     public char getSymbol() {
17         return 'H';
18     }
19
20     @Override
21     public Set<UtilityType> getRequiredUtilities() {
22         return EnumSet.of(UtilityType.ELECTRICITY, UtilityType.WATER, UtilityType.INTERNET);
23     }
24
25     @Override
26     public boolean consumesResource(ResourceType type) {
27         return type == ResourceType.LIFESTYLE;
28     }
29 }

```

Figure 3: The Housing zone.

5.1.3 The cell.utility Sub-Package (Resource Generators)

- **UtilityProvider (Abstract Base Class):** Extends Cell. Defines a shared production capacity and the utility type each provider supplies.
 - Fields: static final int CAPACITY = 100.
 - Methods: getCapacity(), abstract getUtilityType(), isConnectable() (returns false — providers are sources, not carriers).
- **PowerPlant (Concrete Class):** Extends UtilityProvider. Symbol 'P'; supplies UtilityType.ELECTRICITY.
- **WaterStation (Concrete Class):** Extends UtilityProvider. Symbol 'W'; supplies UtilityType.WATER.
- **InternetHub (Concrete Class):** Extends UtilityProvider. Symbol 'T'; supplies UtilityType.INTERNET.

```
1 package cell.utility;
2
3 import enums.UtilityType;
4
5 public class PowerPlant extends UtilityProvider {
6     public PowerPlant(int row, int col) {
7         super(row, col);
8     }
9
10    @Override
11    public char getSymbol() {
12        return 'P';
13    }
14
15    @Override
16    public UtilityType getUtilityType() {
17        return UtilityType.ELECTRICITY;
18    }
19 }
```

Figure 4: The PowerPlant utility provider.

5.1.4 The cell.service Sub-Package (Municipal Services)

- **ServiceProvider (Abstract Base Class):** Extends Cell. Projects a beneficial service across a fixed radius.
 - Fields: final int radius (set via constructor).
 - Methods: getRadius(), abstract getServiceType().
- **Hospital (Concrete Class):** Extends ServiceProvider. Symbol 'D'; radius 3; provides ServiceType.HEALTH.
- **PoliceStation (Concrete Class):** Extends ServiceProvider. Symbol 'F'; radius 5; provides ServiceType.SECURITY.
- **School (Concrete Class):** Extends ServiceProvider. Symbol 'S'; radius 4; provides ServiceType.EDUCATION.

```
1 package cell.service;
2
3 import enums.ServiceType;
4
5 public class Hospital extends ServiceProvider {
6     public Hospital(int row, int col) {
7         super(row, col, 3);
8     }
9
10    @Override
11    public char getSymbol() {
12        return 'D';
13    }
14
15    @Override
16    public ServiceType getServiceType() {
17        return ServiceType.HEALTH;
18    }
19 }
```

Figure 5: The Hospital service provider.

5.1.5 The enums Package (Shared Vocabularies)

- **ResourceType:** POPULATION, GOODS, LIFESTYLE — the economic outputs that zones produce and consume.
- **UtilityType:** ELECTRICITY, WATER, INTERNET — the flows carried by utility providers.
- **ServiceType:** SECURITY, HEALTH, EDUCATION — the radial benefits supplied by service providers.
 - Each constant carries a lowercase `getLabel()` string used directly in console log messages.

5.1.6 The interfaces Package (Behavioral Contracts)

- **Connectable:** Declares a single method, `boolean isConnectable()`, implemented by `Cell`. Roads and zones return true so they carry utility flow during BFS; empty cells, utilities, and services return false.
- **Producible:** Implemented by `Zone`. Declares `Map<ResourceType,Integer> produce()` and `int getOutput(ResourceType type)`, describing what a zone generates each tick.

5.1.7 The city Package (Engine Orchestration)

- **City:** The structural container for the grid. Built through the static factory `City.fromFile(Path)`.
- **Fields:** `Cell[ ][ ]` grid, plus `List<Zone>`, `List<UtilityProvider>`, and `List<ServiceProvider>` populated during loading.
- **Methods:** `fromFile()`, `getCell()`, `isInside()`, `getNeighbors()` (4-directional), `getZonesInRadius()`, and the three accessor lists. A private `createCell()` switch maps each symbol to a concrete type.

- **Simulation:** The heart of the application. Runs the tick loop and writes all output.
- **Fields:** final City city, PrintStream out, EnumMap<ResourceType,Integer> resourcePools.
- **Methods:** run(int ticks), runTick(), distributeServices(), distributeUtilities() (BFS per provider), distributeResources() (even pool sharing), updateZones(), accumulateProduction().

5.1.8 The ObjectVilleGame Entry Point

- The top-level class containing main. It validates command-line arguments, parses the tick count, builds the City from the map file, and runs the Simulation either to System.out or to an optional output file.
- **Usage:** java -jar ObjectVilleGame.jar <map-file> <ticks> [output-file]. Invalid tick counts and bad map files produce clear error messages and non-zero exit codes.

6. Systems Interaction and Feature Flow

6.1 The Tick Loop

Each tick follows a fixed, ordered sequence so that the inputs to one phase are always ready before the next phase runs.

```

40     private void runTick() {
41         for (Zone zone : city.getZones()) {
42             zone.resetTickData();
43         }
44
45         distributeServices();
46         distributeUtilities();
47         distributeResources();
48         updateZones();
49         accumulateProduction();
50     }
51
52     private void distributeServices() {
53         city.getServiceProviders().forEach(provider →
54             city.getZonesInRadius(provider.getRow(), provider.getCol(), provider.getRadius())
55                 .forEach(zone → {
56                     if (usesService(zone, provider.getServiceType())) {
57                         zone.receiveService(provider.getServiceType());
58                         out.println(zoneName(zone) + " at (" + zone.getRow() + ", " + zone.getCol()
59                             + ") received " + provider.getServiceType().getLabel() + " service");
60                     }
61                 })
62     );
63     }

```

Figure 6: The runTick loop and service distribution in Simulation.

6.2 Utility Distribution Algorithm (BFS)

Rather than scanning every coordinate, each utility provider performs a breadth-first search from its own position. The search only steps into cells that report `isConnectable() == true` (roads and zones), so a provider can serve a zone only when a connected path of carriers exists between them. The provider delivers its utility to any zone whose `getRequiredUtilities()` set contains that utility type, up to its fixed capacity of 100 units; once the capacity is exhausted, the search stops.

```
// each provider floods outward through connectable cells
int remaining = provider.getCapacity();
Queue<int[ ]> queue = new ArrayDeque<>();
queue.add(new int[ ]{provider.getRow(), provider.getCol()});
while (!queue.isEmpty() && remaining > 0) {
    int[ ] pos = queue.remove();
    Cell cell = city.getCell(pos[0], pos[1]);
    if (cell instanceof Zone zone
        && zone.getRequiredUtilities().contains(utilityType)) {
        int need = Math.max(0, zone.getDemand()
            - zone.getUtilityAmount(utilityType));
        int amount = Math.min(need, remaining);
        if (amount > 0) {
            zone.receiveUtility(utilityType, amount);
            remaining -= amount;
        }
    }
    for (int[ ] n : city.getNeighbors(pos[0], pos[1])) {
        if (!visited[n[0]][n[1]]) {
            Cell next = city.getCell(n[0], n[1]);
            if (next != null && next.isConnectable()) {
                visited[n[0]][n[1]] = true;
                queue.add(n);
            }
        }
    }
}
}
```

6.3 Service Radius Tracking

Municipal providers project their benefit using straight-line Euclidean distance, computed with `Math.hypot`:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

If the distance between a provider and a zone is within the provider's radius, the zone receives that service. Security is granted to all zone types, while health and education are granted to Housing. Services feed directly into each zone's `calculateTargetLevel()`: for example, a Housing block that has security, health, and education can rise toward level 2, and then to level 3 once it also receives lifestyle resources.

6.4 Resource Pooling and Zone Growth

Each zone produces one resource type: Housing produces population, Industrial produces goods, and Commercial produces lifestyle. Output is added to a shared resource pool at the end of a tick (`accumulateProduction`). On the following tick, `distributeResources` divides each resource type evenly among the zones that consume it (Commercial consumes population and goods; Housing consumes lifestyle; Industrial consumes population). A zone's base output scales with its level and the minimum utility it received, so a fully supplied, fully serviced zone grows and produces more, while a zone missing a required utility drops back to level 0.

7. Results and Simulation Testing

The scenarios below were run against the compiled `ObjectVilleGame.jar`. The map files use the engine's own symbols (P, W, T for power/water/internet; D, S, F for hospital/school/police; H for housing; E for empty), and the console listings are the exact, unedited output produced by the program.

7.1 Scenario A: Connected, Fully-Serviced Grid

A small 3x3 map places a single Housing block (H) at the centre, with all three utility providers directly adjacent to it (PowerPlant above, WaterStation right, InternetHub left) and all three service providers within range below it (Hospital, School, and PoliceStation). Because each provider sits next to the connectable House cell, the BFS reaches it and delivers every required utility, and all three services fall inside their radius.

Map file:

```
E P E
T H W
D S F
```

- **Routing:** Electricity, water, and internet all reach the Housing block. Note that the amount delivered on the first tick is 1, not the provider's full capacity: a zone starts with demand = 1, and the engine delivers only need = demand - alreadyReceived, capped by the provider's remaining capacity (100). The capacity 100 is therefore the provider's ceiling, not the amount a single zone receives.
- **Services and growth:** Because the House receives security, health, and education, hasAllServices() is true, so its target level rises to 2. The zone climbs one level per tick (0 to 1, then 1 to 2) and its population output grows accordingly (1, then 2, then 4). Had any one service been missing, hasAllServices() would be false and the House would remain at level 1.

Actual console output (3 ticks):

```
Tick 1
House at (1,1) received health service
House at (1,1) received education service
House at (1,1) received security service
House at (1,1) received 1 electricity
House at (1,1) received 1 internet
House at (1,1) received 1 water
House at (1,1) generated 1 population
House at (1,1) levels up from 0 to 1
Tick 2
```

```
House at (1,1) received health service
House at (1,1) received education service
House at (1,1) received security service
House at (1,1) received 1 electricity
House at (1,1) received 1 internet
House at (1,1) received 1 water
House at (1,1) generated 2 population
House at (1,1) levels up from 1 to 2
Tick 3
House at (1,1) received health service
House at (1,1) received education service
House at (1,1) received security service
House at (1,1) received 2 electricity
House at (1,1) received 2 internet
House at (1,1) received 2 water
House at (1,1) generated 4 population
```

7.2 Scenario B: Disconnected Infrastructure

A 4x4 map places a WaterStation at (0,0) and a Commercial block at (3,3) with only EmptyCells between them and no Road cells. Because the BFS only steps through connectable cells (roads and zones), the provider's search cannot cross the empty gap, so the isolated zone is never reached.

Map file:

```
W E E E
E E E E
E E E E
E E E C
```

- **Expected behaviour:** No utility reaches the Commercial block. Importantly, the engine logs a utility line only when a positive amount is delivered (the code guards the message with 'if (amount > 0)'), so there is deliberately no 'received 0 electricity' line in the output. The absence of any 'received' line for this zone is itself the evidence that it is starved.
- **Effect:** With no required utilities, hasRequiredUtilities() returns false, so the zone stays at level 0 and produces nothing. The only line printed for it each tick is its zero production.

Actual console output (2 ticks):

```
Tick 1  
Commercial at (3,3) generated 0 lifestyle  
Tick 2  
Commercial at (3,3) generated 0 lifestyle
```

7.3 Official Sample Map Verification

The compiled ObjectVilleGame.jar was also run with the provided map00.txt file for 10 ticks. The produced console output was compared against the provided sample output file and matched exactly line-for-line. This additional verification checks the official-sized map rather than only the smaller Scenario A and Scenario B maps.

Because the official output includes many service assignments, BFS utility deliveries, city-wide resource distributions, level changes, and production records, this comparison confirms that the submitted implementation follows the required simulation order and major ObjectVille II mechanics on the reference data set.

8. Challenges and Solutions

8.1 Serving Only Truly Connected Zones

- **Root cause:** An early version delivered utilities to any zone that required them, regardless of whether a path of carriers existed. This let disconnected zones receive resources they should not have, masking broken layouts.
- **Corrective action:** Distribution was rewritten as a per-provider BFS that only steps through cells where isConnectable() is true. A zone is now served only if it is reachable from the provider through contiguous roads and zones, so isolated sectors correctly receive nothing.

8.2 Capacity Exhaustion and Fair Delivery

- **Root cause:** Without a stopping condition, a provider's loop could keep attempting deliveries after its output was spent, producing inconsistent results.
- **Corrective action:** Each provider tracks a 'remaining' counter initialized to its capacity (100). Every delivery subtracts the amount given, and the BFS halts as soon as remaining reaches zero, guaranteeing that no provider over-allocates.

8.3 Level Stability Across Ticks

- **Root cause:** Zone levels could oscillate if growth and decay were applied in the same step.
- **Corrective action:** Each zone computes a target level from its services and resources, then moves at most one level toward that target per tick (or drops straight to 0 when a required utility is missing), giving smooth, predictable growth.

8.4 Build and Java Version Compatibility

During the final testing phase, we faced a problem while running the ObjectVilleGame.jar file on Windows. The first issue was related to Java version compatibility. The JAR file could not start because it had been compiled with a newer Java version than the Java version installed on the computer. This caused an `UnsupportedClassVersionError` before the program could run.

This problem was not caused by the simulation code or the project logic. The main issue was the difference between the Java version used to build the JAR file and the Java version used to run it. To fix this, we rebuilt the ObjectVilleGame.jar file with a compatible Java version.

We also had another problem with the build.bat file. While testing the project, the project folder had been downloaded more than once from GitHub, so Windows automatically added numbers such as “(4)” to the folder name. Because of this, the project path contained spaces and parentheses, for example `SE116_Project-main (4)`. The old build.bat file did not handle these kinds of paths correctly, and the javac command interpreted the path incorrectly.

At first, we did not notice that the folder name and path format were also causing a build problem, so this issue took longer to solve. After identifying the problem, we updated build.bat so that source file paths are written inside quotation marks. We also added error checking so that the script stops if compilation or JAR creation fails, instead of showing a misleading successful build message.

The `build.sh` file is used for Linux or Mac systems, while build.bat is used for Windows. Both files are included so that the project can be built more easily on different operating systems. After these updates, the project can be run with the following command:

```
java -jar ObjectVilleGame.jar map00.txt 5
```

9. Conclusion and Future Work

ObjectVille II demonstrates how clean object-oriented design solves the core challenges of grid simulation. By isolating cell types into specialized sub-packages, decoupling behavior through the Connectable and Producible interfaces, sharing a common vocabulary via the enums package, and centralizing all cross-cell logic in the City and Simulation classes, the system stays readable and resistant to state-corruption bugs. Utility routing through BFS, radius-based services, and pooled resource sharing together produce a coherent, testable simulation engine.

Future updates could extend this architecture through three major features:

1. **Configurable provider capacities:** Replacing the shared constant capacity with per-provider values to model larger or smaller infrastructure.
2. **Interactive GUI canvas:** Building a visual interface (for example with JavaFX) to display the active grid and animate resource flow in real time.
3. **Dynamic disaster events:** Introducing random breakdowns of providers or roads to evaluate how gracefully the network re-routes resources under stress.

10. Required Report Questions

This section answers the six report questions listed in the ObjectVille II project specification directly, so the implementation discussion can be checked against the grading requirements.

10.1 Use of Inheritance, Polymorphism, Abstract Classes, and Interfaces

Inheritance is used to model the city entities as a shared cell hierarchy. Cell is the abstract parent of every map element, while Zone, UtilityProvider, and ServiceProvider specialize the common Cell behavior for consumers, utility generators, and service buildings. Concrete classes such as Housing, Commercial, Industrial, PowerPlant, WaterStation, InternetHub, PoliceStation, Hospital, School, Road, and EmptyCell inherit from these base classes.

Abstract classes are used where an object category has shared state but should not be instantiated directly. Cell stores coordinates and defines common display behavior; Zone stores level, demand, received utilities, received services, received resources, and produced output; UtilityProvider stores the shared capacity behavior; ServiceProvider stores radius behavior. This

avoids duplicated fields and keeps each concrete subclass focused on its specific symbol, required utilities, consumed resources, and provided type.

Polymorphism is used throughout the engine. City stores cells in a `Cell[ ][ ]` grid and also groups them as `List<Zone>`, `List<UtilityProvider>`, and `List<ServiceProvider>`. Simulation can call methods such as `getRequiredUtilities()`, `getUtilityType()`, `getServiceType()`, `isConnectable()`, `produce()`, and `getSymbol()` through abstract or parent references without manually depending on every concrete type during the main simulation flow.

Interfaces are used to express behavior separately from class identity. `Connectable` marks whether a cell can carry BFS utility flow. `Road` and `Zone` objects return true, while `EmptyCell`, `UtilityProvider`, and `ServiceProvider` return false. `Producible` is implemented by `Zone` to formalize resource production through `produce()` and `getOutput(ResourceType type)`.

10.2 Openness to Extension for New Types

The design is partially open to extension. New concrete classes can be added cleanly because the common behavior is already centralized in abstract classes and interfaces, but the submitted version also uses closed enums and a factory switch, so some central files must still be updated when the vocabulary of the game changes.

If a new zone type such as `Governmental` is introduced, requiring population and producing law for a possible level 4 system, the following parts would need to change:

- Add a new `ResourceType` value such as `LAW` if law becomes a city-wide resource.
- Create a `Governmental` class extending `Zone`, with its own `getSymbol()`, `getRequiredUtilities()`, `consumesResource()`, `produce()`, and `calculateTargetLevel()` behavior.
- Update `City.createCell()` to map a new character symbol to `Governmental`, and make sure `City` adds it to the zones list through the existing `Zone` polymorphism.
- Update `Simulation.producedType()`, `zoneName()`, and resource distribution logic if law is pooled and distributed like population, goods, and lifestyle.
- Update `Zone` level limits and the level-calculation rules if level 4 becomes available to all zones, because the current `Zone` base class clamps levels between 0 and 3.

Therefore, the object hierarchy makes the new class easy to add, but the closed enum vocabulary and level rules still require central edits.

If a new service cell type such as `Park` is introduced to provide a recreational service, the required changes would be smaller. The project would need a new `ServiceType.RECREATION` value, a `Park` class extending `ServiceProvider`, and a new symbol mapping in `City.createCell()`.

The existing `getZonesInRadius()` and `distributeServices()` mechanism would still work because it already iterates over `ServiceProvider` references. Additional changes would only be needed in zone growth logic if recreation affects level requirements.

10.3 Memory-Resident Structures and Generic Types

The project uses several memory-resident structures to keep the simulation state in memory during runtime:

- `Cell[ ][ ]` grid stores the complete map after the input file is parsed.
- `List<Zone>`, `List<UtilityProvider>`, and `List<ServiceProvider>` keep typed groups for efficient simulation iteration.
- `EnumMap<UtilityType, Integer>`, `EnumMap<ResourceType, Integer>`, and `EnumSet<ServiceType>` store each zone's per-tick received utilities, resources, and services.
- `EnumMap<ResourceType, Integer>` `resourcePools` stores city-wide pooled production between ticks.
- `Queue<int[ ]>` implemented with `ArrayDeque<int[ ]>` and `boolean[ ][ ]` `visited` support BFS utility propagation from each provider.
- `Map<ResourceType, Integer>`, `Set<UtilityType>`, and other generic collections are used to return production data and required utility sets safely.

10.4 Lambdas and Packaged Units

Packaged units are used extensively. The code is separated into `city`, `cell`, `cell.zone`, `cell.utility`, `cell.service`, `enums`, and `interfaces` packages. This package structure keeps orchestration, domain objects, shared vocabularies, and behavioral contracts separate.

Lambda expressions are used in `Simulation.distributeServices()`, where service providers and zones within radius are processed with `forEach` calls. The code also uses method references and streams in `Zone`, such as `produced.values().stream().mapToInt(Integer::intValue).sum()` and `servicesReceived.stream().map(Enum::name).collect(Collectors.joining(", "))`. Switch expressions are used in `City.createCell()` to construct the correct concrete object from each map symbol.

10.5 Map File Handling

The map is handled by `City.fromFile(Path)`. The method reads all lines from the map file, strips whitespace, ignores blank lines and comment lines beginning with `#`, and supports an optional first line containing row and column counts. Each remaining row is parsed either as whitespace-separated symbols or as a continuous character row.

The loader validates that the map is not empty, that all rows have the same number of columns, and that every symbol is known. `City.createCell()` then converts each character into the correct concrete `Cell` subclass. While loading, the city also populates separate zone, utility-provider, and service-provider lists, which makes the later simulation phases faster and cleaner.

10.6 Postmortem: What Went Right and What Went Wrong

What went right: the inheritance structure made the implementation easier to divide among team members because each class had a clear responsibility. The BFS utility distribution worked well after the `Connectable` interface separated roads and zones from empty cells and providers. `EnumMap` and `EnumSet` also made resource, utility, and service tracking safer than using raw strings. What went wrong: early implementations risked giving utilities to disconnected zones or over-allocating provider capacity. Another common pitfall was level instability, where a zone could grow and shrink too aggressively if all updates were not ordered carefully.

Recovery: the team moved utility distribution into a per-provider BFS with a visited matrix and a remaining-capacity counter. The tick loop was also ordered consistently: reset tick data, distribute services, distribute utilities, distribute previous production, update zones, and then accumulate new production for the next tick. This made the simulation output predictable and easier to compare with the required sample output.

11. References

1. İzmir University of Economics, Department of Software Engineering, SE116 Project Specifications Handbook, Spring 2025-2026.
2. Deitel, P., & Deitel, H. (2017). Java How to Program, Early Objects (11th ed.). Pearson.
3. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.